

Chapter 5. Cognitive Architectures with LangGraph

So far, we've looked at the most common features of LLM applications:

- Prompting techniques in the [Preface](#) and [Chapter 1](#)
- RAG in Chapters [2](#) and [3](#)
- Memory in [Chapter 4](#)

The next question should be: How do we assemble these pieces into a coherent application that achieves the goal we set out to solve? To draw a parallel with the world of bricks and mortar, a swimming pool and a one-story house are built of the same materials, but obviously serve very different purposes. What makes them uniquely suited to their different purposes is the plan for how those materials are combined—that is, their architecture. The same is true when building LLM applications. The most important decisions you have to make are how to assemble the different components you have at your disposal (such as RAG, prompting techniques, memory) into something that achieves your purpose.

Before we look at specific architectures, let's walk through an example. Any LLM application you might build will start from a purpose: what the app is designed to do. Let's say you want to build an email assistant—an LLM application that reads your emails before you do and aims to reduce the amount of emails you need to look at. The application might do this by archiving a few uninteresting ones, directly replying to some, and marking others as deserving of your attention later.

You probably also would want the app to be bound by some constraints in its action. Listing those constraints helps tremendously, as they will help inform the search for the right architecture. [Chapter 8](#) covers these constraints in more detail and how to work with them. For this hypothetical email assistant, let's say we'd like it to do the following:

- Minimize the number of times it interrupts you (after all, the whole point is to save time).
- Avoid having your email correspondents receive a reply that you'd never have sent yourself.

This hints at the key trade-off often faced when building LLM apps: the trade-off between *agency* (or the capacity to act autonomously) and *reliability* (or the degree to which you can trust its outputs). Intuitively, the email assistant will be more useful if it takes more actions without your involvement, but if you take it too far, it will inevitably send emails you wish it hadn't.

One way to describe the degree of autonomy of an LLM application is to evaluate how much of the behavior of the application is determined by an LLM (versus code):

- Have an LLM decide the output of a step (for instance, write a draft reply to an email).
- Have an LLM decide the next step to take (for instance, for a new email, decide between the three actions it can take on an email: archive, reply, or mark for review).
- Have an LLM decide what steps are available to take (for instance, have the LLM write code that executes a dynamic action you didn't preprogram into the application).

We can classify a number of popular *recipes* for building LLM applications based on where they fall in this spectrum of autonomy, that is, which of the three tasks just mentioned are handled by an LLM and which remain in the hands of the developer or user. These recipes can be called *cognitive architectures*. In the artificial intelligence field, the term *cognitive architecture* has long been used to denote models of human reasoning (and their implementations in computers). An LLM cognitive architecture (the term was first applied to LLMs, to our knowledge, in a paper¹) can be defined as a recipe for the steps to be taken by an LLM application (see [Figure 5-1](#)). A *step* is, for instance, retrieval of relevant documents (RAG), or calling an LLM with a chain-of-thought prompt.

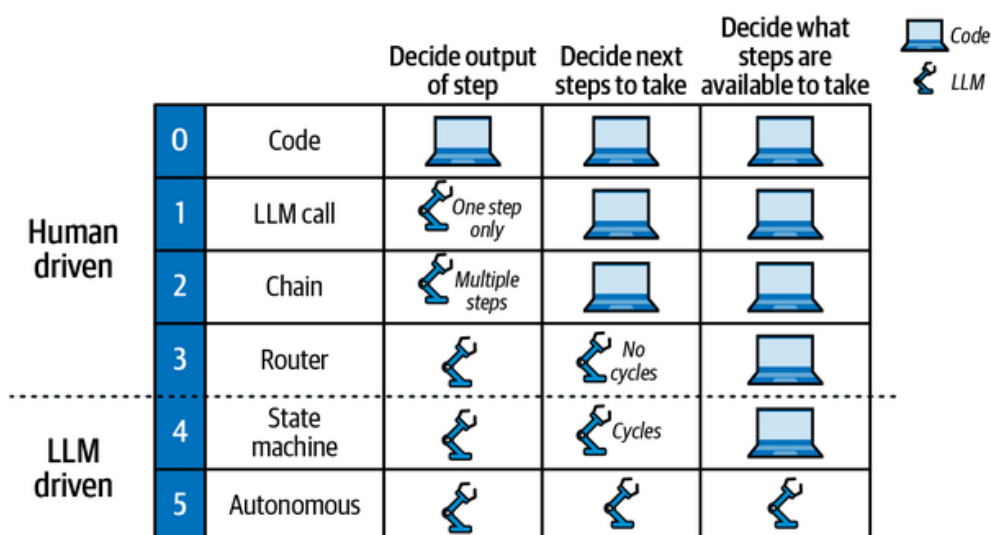


Figure 5-1. Cognitive architectures for LLM applications

Now let's look at each of the major architectures, or recipes, that you can use when building your application (as shown in [Figure 5-1](#)):

0: Code

This is not an LLM cognitive architecture (hence we numbered it **0**), as it doesn't use LLMs at all. You can think of this as regular software you're used to writing. The first interesting architecture (for this book, at any rate) is actually the next one.

1: LLM call

This is the majority of the examples we've seen in the book so far, with one LLM call only. This is useful mostly when it's part of a larger application that makes use of an LLM for achieving a specific task, such as translating or summarizing a piece of text.

2: Chain

The next level up, so to speak, comes with the use of multiple LLM calls in a predefined sequence. For instance, a text-to-SQL application (which receives as input from the user a natural language description of some calculation to make over a database) could make use of two LLM calls in sequence:

One LLM call to generate a SQL query, from the natural language query, provided by the user, and a description of the database contents, provided by the developer.

And another LLM call to write an explanation of the query appropriate for a nontechnical user, given the query generated in the previous call. This one could then be used to enable the user to check if the generated query matches his request.

3: Router

This next step comes from using the LLM to define the sequence of steps to take. That is, whereas the chain architecture always executes a static sequence of steps (however many) determined by the developer, the router architecture is characterized by using an LLM to choose between certain predefined steps. An example would be a RAG application with multiple indexes of documents from different domains, with the following steps:

1. An LLM call to pick which of the available indexes to use, given the user-supplied query and the developer-supplied description of the indexes.

2. A retrieval step that queries the chosen index for the most relevant documents for the user query.
3. Another LLM call to generate an answer, given the user-supplied query and the list of relevant documents fetched from the index.

That's as far as we'll go in this chapter. We will talk about each of these architectures in turn. The next chapters discuss the agentic architectures, which make even more use of LLMs. But first let's talk about some better tooling to help us on this journey.

Architecture #1: LLM Call

As an example of the LLM call architecture, we'll return to the chatbot we created in [Chapter 4](#). This chatbot will respond directly to user messages.

Start by creating a `StateGraph`, to which we'll add a node to represent the LLM call:

Python

```
from typing import Annotated, TypedDict

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI

model = ChatOpenAI()

class State(TypedDict):
    # Messages have the type "list". The `add_messages`
    # function in the annotation defines how this state should
    # be updated (in this case, it appends new messages to the
    # list, rather than replacing the previous messages)
    messages: Annotated[list, add_messages]

def chatbot(state: State):
    answer = model.invoke(state["messages"])
    return {"messages": [answer]}

builder = StateGraph(State)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, 'chatbot')
builder.add_edge('chatbot', END)

graph = builder.compile()
```

```

import {
  StateGraph,
  Annotation,
  messagesStateReducer,
  START, END
} from '@langchain/langgraph'
import {ChatOpenAI} from '@langchain/openai'

const model = new ChatOpenAI()

const State = {
  /**
   * The State defines three things:
   * 1. The structure of the graph's state (which "channels" are available
   * read/write)
   * 2. The default values for the state's channels
   * 3. The reducers for the state's channels. Reducers are functions that
   * determine how to apply updates to the state. Below, new messages are
   * appended to the messages array.
   */
  messages: Annotation({
    reducer: messagesStateReducer,
    default: () => []
  }),
}

async function chatbot(state) {
  const answer = await model.invoke(state.messages)
  return {"messages": answer}
}

const builder = new StateGraph(State)
  .addNode('chatbot', chatbot)
  .addEdge(START, 'chatbot')
  .addEdge('chatbot', END)

const graph = builder.compile()

```

We can also draw a visual representation of the graph:

Python

```
graph.get_graph().draw_mermaid_png()
```

JavaScript

```
await graph.getGraph().drawMermaidPng()
```

The graph we just made looks like [Figure 5-2](#).

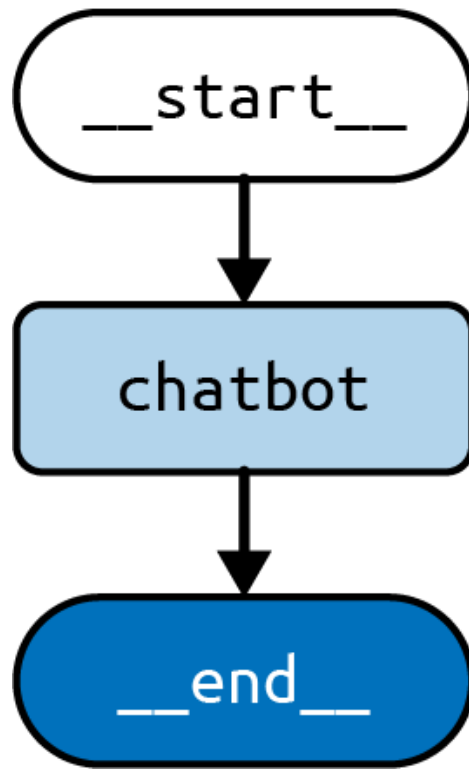


Figure 5-2. The LLM call architecture

You can run it with the familiar `stream()` method you've seen in earlier chapters:

Python

```
input = {"messages": [HumanMessage('hi!)]}
for chunk in graph.stream(input):
    print(chunk)
```

JavaScript

```
const input = {messages: [new HumanMessage('hi!)]}
for await (const chunk of await graph.stream(input)) {
  console.log(chunk)
}
```

The output:

```
{ "chatbot": { "messages": [AIMessage("How can I help you?")] } }
```

Notice how the input to the graph was in the same shape as the `State` object we defined earlier; that is, we sent in a list of messages in the `messages` key of a dictionary.

This is the simplest possible architecture for using an LLM, which is not to say that it should never be used. Here are some examples of where you might see it in action in popular products, among many others:

- AI-powered features such as summarize and translate (such as you can find in Notion, a popular writing software) can be powered by a single LLM call.
- Simple SQL query generation can be powered by a single LLM call, depending on the UX and target user the developer has in mind.

Architecture #2: Chain

This next architecture extends on all that by using multiple LLM calls, in a predefined sequence (that is, different invocations of the application do the same sequence of LLM calls, albeit with different inputs and results).

Let's take as an example a text-to-SQL application, which receives as input from the user a natural language description of some calculation to make over a database. We mentioned earlier that this could be achieved with a single LLM call, to generate a SQL query, but we can create a more sophisticated application by making use of multiple LLM calls in sequence. Some authors call this architecture *flow engineering*.²

First let's describe the flow in words:

1. One LLM call to generate a SQL query from the natural language query, provided by the user, and a description of the database contents, provided by the developer.
2. Another LLM call to write an explanation of the query appropriate for a nontechnical user, given the query generated in the previous call. This one could then be used to enable the user to check if the generated query matches his request.

You could also extend this even further (but we won't do that here) with additional steps to be taken after the preceding two:

3. Executes the query against the database, which returns a two-dimensional table.
4. Uses a third LLM call to summarize the query results into a textual answer to the original user question.

And now let's implement this with LangGraph:

Python

```

from typing import Annotated, TypedDict

from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai import ChatOpenAI

from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages

# useful to generate SQL query
model_low_temp = ChatOpenAI(temperature=0.1)
# useful to generate natural language outputs
model_high_temp = ChatOpenAI(temperature=0.7)

class State(TypedDict):
    # to track conversation history
    messages: Annotated[list, add_messages]
    # input
    user_query: str
    # output
    sql_query: str
    sql_explanation: str

class Input(TypedDict):
    user_query: str

class Output(TypedDict):
    sql_query: str
    sql_explanation: str

generate_prompt = SystemMessage(
    """You are a helpful data analyst who generates SQL queries for users
    on their questions."""
)

def generate_sql(state: State) -> State:
    user_message = HumanMessage(state["user_query"])
    messages = [generate_prompt, *state["messages"], user_message]
    res = model_low_temp.invoke(messages)
    return {
        "sql_query": res.content,
        # update conversation history
        "messages": [user_message, res],
    }

explain_prompt = SystemMessage(
    "You are a helpful data analyst who explains SQL queries to users."
)

def explain_sql(state: State) -> State:
    messages = [
        explain_prompt,
        # contains user's query and SQL query from prev step

```

```

        *state["messages"],
    ]
    res = model_high_temp.invoke(messages)
    return {
        "sql_explanation": res.content,
        # update conversation history
        "messages": res,
    }

builder = StateGraph(State, input=Input, output=Output)
builder.add_node("generate_sql", generate_sql)
builder.add_node("explain_sql", explain_sql)
builder.add_edge(START, "generate_sql")
builder.add_edge("generate_sql", "explain_sql")
builder.add_edge("explain_sql", END)

graph = builder.compile()

```

JavaScript

```

import {
    HumanMessage,
    SystemMessage
} from "@langchain/core/messages";
import { ChatOpenAI } from "@langchain/openai";
import {
    StateGraph,
    Annotation,
    messagesStateReducer,
    START,
    END,
} from "@langchain/langgraph";

// useful to generate SQL query
const modelLowTemp = new ChatOpenAI({ temperature: 0.1 });
// useful to generate natural language outputs
const modelHighTemp = new ChatOpenAI({ temperature: 0.7 });

const annotation = Annotation.Root({
    messages: Annotation({ reducer: messagesStateReducer, default: () => [] }),
    user_query: Annotation(),
    sql_query: Annotation(),
    sql_explanation: Annotation(),
});

const generatePrompt = new SystemMessage(
    `You are a helpful data analyst who generates SQL queries for users base
    their questions.`
);

async function generateSql(state) {

```

```

const userMessage = new HumanMessage(state.user_query);
const messages = [generatePrompt, ...state.messages, userMessage];
const res = await modelLowTemp.invoke(messages);
return {
  sql_query: res.content as string,
  // update conversation history
  messages: [userMessage, res],
};
}

const explainPrompt = new SystemMessage(
  "You are a helpful data analyst who explains SQL queries to users."
);

async function explainSql(state) {
  const messages = [explainPrompt, ...state.messages];
  const res = await modelHighTemp.invoke(messages);
  return {
    sql_explanation: res.content as string,
    // update conversation history
    messages: res,
  };
}

const builder = new StateGraph(annotation)
  .addNode("generate_sql", generateSql)
  .addNode("explain_sql", explainSql)
  .addEdge(START, "generate_sql")
  .addEdge("generate_sql", "explain_sql")
  .addEdge("explain_sql", END);

const graph = builder.compile();

```

The visual representation of the graph is shown in [Figure 5-3](#).

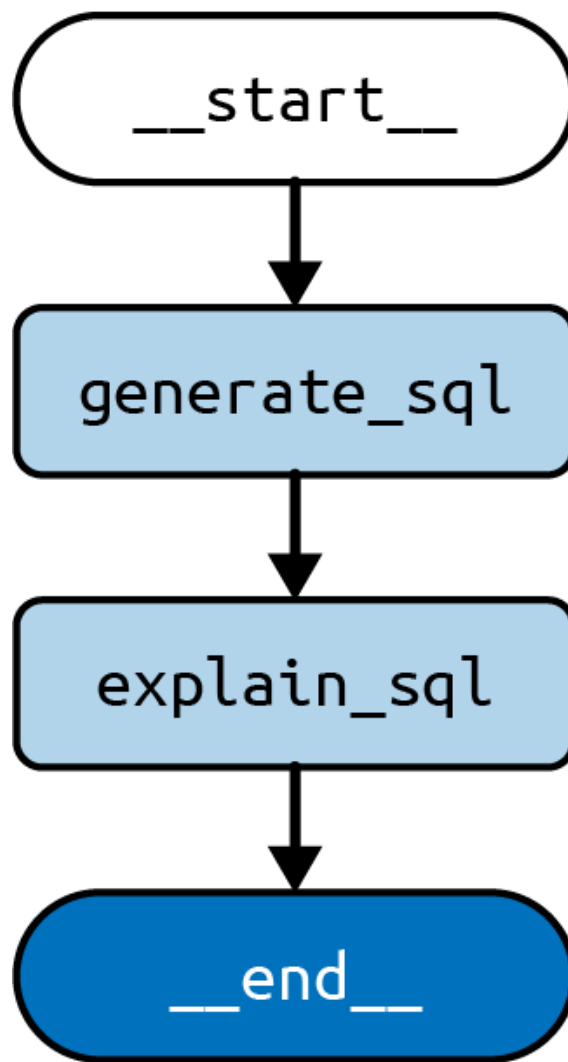


Figure 5-3. The chain architecture

Here's an example of inputs and outputs:

Python

```
graph.invoke({
    "user_query": "What is the total sales for each product?"
})
```

JavaScript

```
await graph.invoke({
  user_query: "What is the total sales for each product?"
})
```

The output:

```
{
  "sql_query": "SELECT product_name, SUM(sales_amount) AS total_sales\nFROM\n  sales\nGROUP BY product_name;",
  "sql_explanation": "This query will retrieve the total sales for each pr\n                    by summing up the sales_amount column for each product and grouping
```

```
        results by product_name.",
    }
```

First, the `generate_sql` node is executed, which populates the `sql_query` key in the state (which will be part of the final output) and updates the `messages` key with the new messages. Then the `explain_sql` node runs, taking the SQL query generated in the previous step and populating the `sql_explanation` key in the state. At this point, the graph finishes running, and the output is returned to the caller.

Note also the use of separate input and output schemas when creating the `StateGraph`. This lets you customize which parts of the state are accepted as input from the user and which are returned as the final output. The remaining state keys are used by the graph nodes internally to keep intermediate state and are made available to the user as part of the streaming output produced by `stream()`.

Architecture #3: Router

This next architecture moves up the autonomy ladder by assigning to LLMs the next of the responsibilities we outlined before: deciding the next step to take. That is, whereas the chain architecture always executes a static sequence of steps (however many), the router architecture is characterized by using an LLM to choose between certain predefined steps.

Let's use the example of a RAG application with access to multiple indexes of documents from different domains (refer to [Chapter 2](#) for more on indexing). Usually you can extract better performance from LLMs by avoiding the inclusion of irrelevant information in the prompt. Therefore, in building this application, we should try to pick the right index to use for each query and use only that one. The key development in this architecture is to use an LLM to make this decision, effectively using an LLM to evaluate each incoming query and decide which index it should use for that *particular* query.

NOTE

Before the advent of LLMs, the usual way of solving this problem would be to build a classifier model using ML techniques and a dataset mapping example user queries to the right index. This could prove quite challenging, as it requires the following:

- Assembling that dataset by hand
- Generating enough *features* (quantitative attributes) from each user query to enable training a classifier for the task

LLMs, given their encoding of human language, can effectively serve as this classifier with zero, or very few, examples or additional training.

First, let's describe the flow in words:

1. An LLM call to pick which of the available indexes to use, given the user-supplied query, and the developer-supplied description of the indexes
2. A retrieval step that queries the chosen index for the most relevant documents for the user query
3. Another LLM call to generate an answer, given the user-supplied query and the list of relevant documents fetched from the index

And now let's implement it with LangGraph:

Python

```
from typing import Annotated, Literal, TypedDict

from langchain_core.documents import Document
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.vectorstores.in_memory import InMemoryVectorStore
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages

embeddings = OpenAIEmbeddings()
# useful to generate SQL query
model_low_temp = ChatOpenAI(temperature=0.1)
# useful to generate natural language outputs
model_high_temp = ChatOpenAI(temperature=0.7)

class State(TypedDict):
    # to track conversation history
    messages: Annotated[list, add_messages]
    # input
    user_query: str
```

```

        # output
        domain: Literal["records", "insurance"]
        documents: list[Document]
        answer: str

class Input(TypedDict):
    user_query: str

class Output(TypedDict):
    documents: list[Document]
    answer: str

# refer to Chapter 2 on how to fill a vector store with documents
medical_records_store = InMemoryVectorStore.from_documents([], embeddings)
medical_records_retriever = medical_records_store.as_retriever()

insurance_faqs_store = InMemoryVectorStore.from_documents([], embeddings)
insurance_faqs_retriever = insurance_faqs_store.as_retriever()

router_prompt = SystemMessage(
    """You need to decide which domain to route the user query to. You have
    domains to choose from:
        - records: contains medical records of the patient, such as
        diagnosis, treatment, and prescriptions.
        - insurance: contains frequently asked questions about insurance
        policies, claims, and coverage.

    Output only the domain name."""
)

def router_node(state: State) -> State:
    user_message = HumanMessage(state["user_query"])
    messages = [router_prompt, *state["messages"], user_message]
    res = model_low_temp.invoke(messages)
    return {
        "domain": res.content,
        # update conversation history
        "messages": [user_message, res],
    }

def pick_retriever(
    state: State,
) -> Literal["retrieve_medical_records", "retrieve_insurance_faqs"]:
    if state["domain"] == "records":
        return "retrieve_medical_records"
    else:
        return "retrieve_insurance_faqs"

def retrieve_medical_records(state: State) -> State:
    documents = medical_records_retriever.invoke(state["user_query"])
    return {
        "documents": documents,
    }

```

```

def retrieve_insurance_faqs(state: State) -> State:
    documents = insurance_faqs_retriever.invoke(state["user_query"])
    return {
        "documents": documents,
    }

medical_records_prompt = SystemMessage(
    """You are a helpful medical chatbot who answers questions based on the
    patient's medical records, such as diagnosis, treatment, and
    prescriptions."""
)

insurance_faqs_prompt = SystemMessage(
    """You are a helpful medical insurance chatbot who answers frequently
    questions about insurance policies, claims, and coverage."""
)

def generate_answer(state: State) -> State:
    if state["domain"] == "records":
        prompt = medical_records_prompt
    else:
        prompt = insurance_faqs_prompt
    messages = [
        prompt,
        *state["messages"],
        HumanMessage(f"Documents: {state['documents']}"),
    ]
    res = model_high_temp.invoke(messages)
    return {
        "answer": res.content,
        # update conversation history
        "messages": res,
    }

builder = StateGraph(State, input=Input, output=Output)
builder.add_node("router", router_node)
builder.add_node("retrieve_medical_records", retrieve_medical_records)
builder.add_node("retrieve_insurance_faqs", retrieve_insurance_faqs)
builder.add_node("generate_answer", generate_answer)
builder.add_edge(START, "router")
builder.add_conditional_edges("router", pick_retriever)
builder.add_edge("retrieve_medical_records", "generate_answer")
builder.add_edge("retrieve_insurance_faqs", "generate_answer")
builder.add_edge("generate_answer", END)

graph = builder.compile()

```

```

import {
    HumanMessage,
    SystemMessage
} from "@langchain/core/messages";
import {
    ChatOpenAI,
    OpenAIEmbeddings
} from "@langchain/openai";
import {
    MemoryVectorStore
} from "langchain/vectorstores/memory";
import {
    DocumentInterface
} from "@langchain/core/documents";
import {
    StateGraph,
    Annotation,
    messagesStateReducer,
    START,
    END,
} from "@langchain/langgraph";

const embeddings = new OpenAIEmbeddings();
// useful to generate SQL query
const modelLowTemp = new ChatOpenAI({ temperature: 0.1 });
// useful to generate natural language outputs
const modelHighTemp = new ChatOpenAI({ temperature: 0.7 });

const annotation = Annotation.Root({
    messages: Annotation({ reducer: messagesStateReducer, default: () => [] },
    user_query: Annotation(),
    domain: Annotation(),
    documents: Annotation(),
    answer: Annotation(),
});

// refer to Chapter 2 on how to fill a vector store with documents
const medicalRecordsStore = await MemoryVectorStore.fromDocuments(
    [],
    embeddings
);
const medicalRecordsRetriever = medicalRecordsStore.asRetriever();

const insuranceFaqStore = await MemoryVectorStore.fromDocuments(
    [],
    embeddings
);
const insuranceFaqRetriever = insuranceFaqStore.asRetriever();

const routerPrompt = new SystemMessage(
    `You need to decide which domain to route the user query to. You have two
    domains to choose from:

```

- records: contains medical records of the patient, such as diagnosis, treatment, and prescriptions.
- insurance: contains frequently asked questions about insurance policies, claims, and coverage.

Output only the domain name.`

```
);

async function routerNode(state) {
  const userMessage = new HumanMessage(state.user_query);
  const messages = [routerPrompt, ...state.messages, userMessage];
  const res = await modelLowTemp.invoke(messages);
  return {
    domain: res.content as "records" | "insurance",
    // update conversation history
    messages: [userMessage, res],
  };
}

function pickRetriever(state) {
  if (state.domain === "records") {
    return "retrieve_medical_records";
  } else {
    return "retrieve_insurance_faqs";
  }
}

async function retrieveMedicalRecords(state) {
  const documents = await medicalRecordsRetriever.invoke(state.user_query);
  return {
    documents,
  };
}

async function retrieveInsuranceFaqs(state) {
  const documents = await insuranceFaqsRetriever.invoke(state.user_query);
  return {
    documents,
  };
}

const medicalRecordsPrompt = new SystemMessage(
  `You are a helpful medical chatbot who answers questions based on the
  patient's medical records, such as diagnosis, treatment, and
  prescriptions.`
);

const insuranceFaqsPrompt = new SystemMessage(
  `You are a helpful medical insurance chatbot who answers frequently asked
  questions about insurance policies, claims, and coverage.`
);

async function generateAnswer(state) {
```

```

const prompt =
  state.domain === "records" ? medicalRecordsPrompt : insuranceFaqsPrompt
const messages = [
  prompt,
  ...state.messages,
  new HumanMessage(`Documents: ${state.documents}`),
];
const res = await modelHighTemp.invoke(messages);
return {
  answer: res.content as string,
  // update conversation history
  messages: res,
};
}

const builder = new StateGraph(annotation)
  .addNode("router", routerNode)
  .addNode("retrieve_medical_records", retrieveMedicalRecords)
  .addNode("retrieve_insurance_faqs", retrieveInsuranceFaqs)
  .addNode("generate_answer", generateAnswer)
  .addEdge(START, "router")
  .addConditionalEdges("router", pickRetriever)
  .addEdge("retrieve_medical_records", "generate_answer")
  .addEdge("retrieve_insurance_faqs", "generate_answer")
  .addEdge("generate_answer", END);

const graph = builder.compile();

```

The visual representation is shown in [Figure 5-4](#).

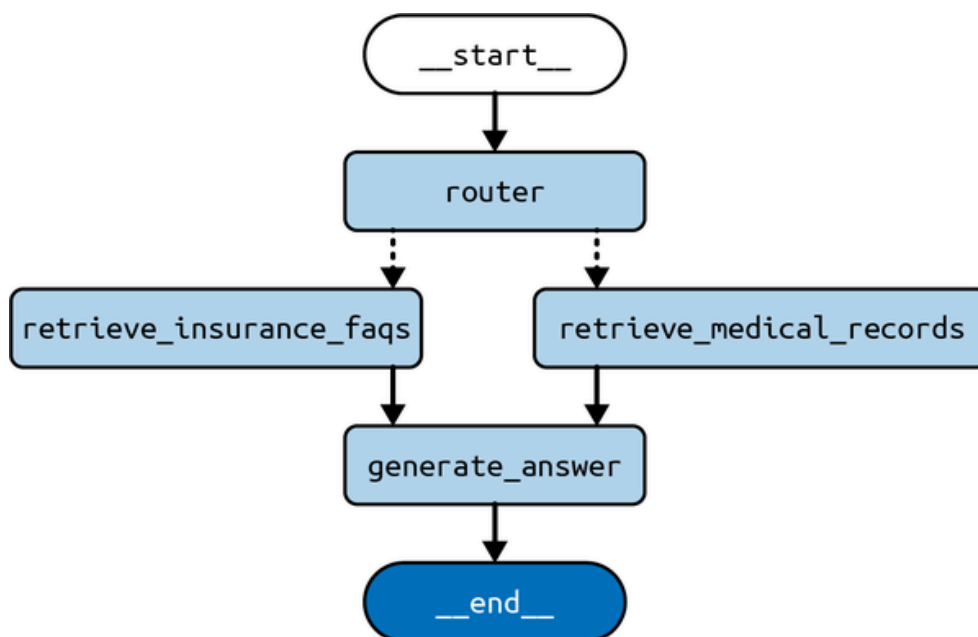


Figure 5-4. The router architecture

Notice how this is now starting to become more useful, as it shows the two possible paths through the graph, through `retrieve_medical_records` or through `retrieve_insurance_faqs`,

and that for both of those, we first visit the `router` node and finish by visiting the `generate_answer` node. These two possible paths were implemented through the use of a conditional edge, implemented in the function `pick_retriever`, which maps the `domain` picked by the LLM to one of the two nodes mentioned earlier. The conditional edge is shown in [Figure 5-4](#) as dotted lines from the source node to the destination nodes.

And now for example inputs and outputs, this time with streaming output:

Python

```
input = {
    "user_query": "Am I covered for COVID-19 treatment?"
}
for c in graph.stream(input):
    print(c)
```

JavaScript

```
const input = {
  user_query: "Am I covered for COVID-19 treatment?"
}
for await (const chunk of await graph.stream(input)) {
  console.log(chunk)
}
```

The output (the actual answer is not shown, since it would depend on your documents):

```
{
  "router": {
    "messages": [
      HumanMessage(content="Am I covered for COVID-19 treatment?"),
      AIMessage(content="insurance"),
    ],
    "domain": "insurance",
  }
}
{
  "retrieve_insurance_faqs": {
    "documents": [...]
  }
}
{
  "generate_answer": {
    "messages": AIMessage(
```

```
        content="...",
    ),
    "answer": "...",
}
}
```

This output stream contains the values returned by each node that ran during this execution of the graph. Let's take it one at a time. The top-level key in each dictionary is the name of the node, and the value for that key is what that node returned:

1. The `router` node returned an update to `messages` (this would allow us to easily continue this conversation using the memory technique described earlier), and the `domain` the LLM picked for this user's query, in this case `insurance`.
2. Then the `pick_retriever` function ran and returned the name of the next node to run, based on the `domain` identified by the LLM call in the previous step.
3. Then the `retrieve_insurance_faqs` node ran, returning a set of relevant documents from that index. This means that on the drawing of the graph seen earlier, we took the left path, as decided by the LLM.
4. Finally, the `generate_answer` node ran, which took those documents and the original user query and produced an answer to the question, which was written to the state (along with a final update to the `messages` key).

Summary

This chapter talked about the key trade-off when building LLM applications: agency versus oversight. The more autonomous an LLM application is, the more it can do—but that raises the need for more mechanisms of control over its actions. We moved on to different cognitive architectures that strike different balances between agency and oversight.

[Chapter 6](#) talks about the most powerful of the cognitive architectures we've seen so far: the agent architecture.

¹ Theodore R. Sumers et al., [“Cognitive Architectures for Language Agents”](#), arXiv, September 5, 2023, updated March 15, 2024.

² Tal Ridnik et al., [“Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering”](#), arXiv, January 16, 2024.

